Ilies NASR
Ismaël RADOUANE
Raphaël LAURENT
Elias MOUSSA

Projet Base De Données 2 – Livrable 3

2025-2026

Création et remplissage de la base

Prompt utilisé :

Tu es un générateur de données SQL pour une base de données artistique appelée ArtConnect. Génère des INSERT INTO cohérents et réalistes pour les tables suivantes : Artist, CommunityMember, Discipline, Artist_Discipline, Gallery, Exhibition, Artwork, ArtworkTag, Artwork_Tag, Artwork_Exhibition, Workshop, Booking, Review.
Contraintes :
- 8 artistes de styles et époques variés (peinture, sculpture, photographie, art numérique)
- 5 galeries dans des villes différentes
- 10 œuvres avec des statuts variés (FOR_SALE, SOLD, EXHIBITED)
- 6 expositions réparties dans les galeries
- 5 ateliers avec niveaux variés
- 10 membres de la communauté
- Des réservations croisées (plusieurs membres, plusieurs ateliers)
- Des avis sur plusieurs œuvres
- Respecte les clés étrangères et les contraintes UNIQUE Réponds uniquement avec le script SQL, sans commentaire ni balise markdown.

Voir github pour fichier insertion SQL

Vues et indexes

Vues :

```sql
-- VUE 1 : Catalogue public des œuvres (masque le prix des œuvres SOLD)
-- Objectif : sécurité / masquage d'attributs sensibles
CREATE OR REPLACE VIEW v_catalogue_public AS
SELECT
  a.id_artwork,
  a.title                                        AS oeuvre,
  ar.name                                        AS artiste,
  a.type,
  a.medium,
  CASE WHEN a.status = 'SOLD' THEN NULL
       ELSE a.price END                          AS prix,
  a.status
FROM Artwork a
JOIN Artist ar ON a.id_artist = ar.id_artist;
```

```sql
-- VUE 2 : Programme des expositions avec galerie
-- Objectif : simplification de requête pour l'interface Exhibitions
CREATE OR REPLACE VIEW v_programme_expositions AS
SELECT
  e.id_exhibition,
  e.title                                          AS exposition,
  e.theme,
  e.startDate,
  e.endDate,
  g.name                                           AS galerie,
  g.address                                        AS adresse,
  g.rating                                         AS note_galerie,
  e.curatorName                                    AS curateur,
  COUNT(ae.id_artwork)                             AS nb_oeuvres
FROM Exhibition e
JOIN Gallery g        ON e.id_gallery    = g.id_gallery
LEFT JOIN Artwork_Exhibition ae ON e.id_exhibition = ae.id_exhibition
GROUP BY e.id_exhibition, e.title, e.theme, e.startDate, e.endDate,
         g.name, g.address, g.rating, e.curatorName;
```

```sql
-- VUE 3 : Tableau de bord des ateliers (places restantes en temps réel)
-- Objectif : requête métier complexe réutilisable
CREATE OR REPLACE VIEW v_ateliers_disponibilite AS
SELECT
  w.id_workshop,
  w.title                                          AS atelier,
  ar.name                                          AS instructeur,
  w.date,
  w.level,
  w.price,
  w.maxParticipants,
  COUNT(b.id_booking)                              AS inscrits,
  (w.maxParticipants - COUNT(b.id_booking))        AS places_restantes
FROM Workshop w
JOIN Artist ar ON w.id_artist = ar.id_artist
LEFT JOIN Booking b ON w.id_workshop = b.id_workshop
  AND b.paymentStatus != 'CANCELLED'
GROUP BY w.id_workshop, w.title, ar.name, w.date,
         w.level, w.price, w.maxParticipants;
```

```sql
-- VUE 4 : Profil artiste enrichi (œuvres, disciplines, note moyenne)
-- Objectif : agrégation pour la page Artists
CREATE OR REPLACE VIEW v_profil_artiste AS
SELECT
  ar.id_artist,
  ar.name,
  ar.city,
  ar.birthYear,
  GROUP_CONCAT(DISTINCT d.name ORDER BY d.name SEPARATOR ', ') AS disciplines,
  COUNT(DISTINCT a.id_artwork)                        AS nb_oeuvres,
  ROUND(AVG(r.rating), 1)                             AS note_moyenne
FROM Artist ar
LEFT JOIN Artist_Discipline ad ON ar.id_artist  = ad.id_artist
LEFT JOIN Discipline d         ON ad.id_discipline = d.id_discipline
LEFT JOIN Artwork a            ON ar.id_artist  = a.id_artist
LEFT JOIN Review r             ON a.id_artwork  = r.id_artwork
GROUP BY ar.id_artist, ar.name, ar.city, ar.birthYear;
```

Indexes :

```sql
-- INDEX 1 : Recherche d'artistes par ville (filtre fréquent dans l'UI)
CREATE INDEX idx_artist_city
  ON Artist(city);

-- INDEX 2 : Recherche d'artiste par nom (barre de recherche)
CREATE INDEX idx_artist_name
  ON Artist(name);

-- INDEX 3 : Filtrage des œuvres par statut (FOR_SALE / SOLD / EXHIBITED)
CREATE INDEX idx_artwork_status
  ON Artwork(status);

-- INDEX 4 : Filtrage des œuvres par artiste (jointure fréquente)
CREATE INDEX idx_artwork_artist
  ON Artwork(id_artist);

-- INDEX 5 : Recherche de réservations par membre
CREATE INDEX idx_booking_member
  ON Booking(id_member);

-- INDEX 6 : Expositions par date de début (tri chronologique)
CREATE INDEX idx_exhibition_startdate
  ON Exhibition(startDate);
```

Justification des choix des indexes :

- idx_artist_city ET idx_artist_name :
- idx_artwork_status : le filtre FOR_SALE/SOLD/EXHIBITED est la requête la plus fréquente sur Artwork.
- idx_artwork_artist : toutes les jointures de Artwork vers Artist utilisent cette colonne.
- idx_booking_member : vérifier les réservations d'un membre est critique pour éviter les doublons.
- idx _exhibition_startdate : le programme des expositions est trié par date.

Déclencheurs et programmes stockés

Déclencheurs :

```
-- TRIGGER 1 : Vérifier que endDate > startDate pour une exposition
CREATE TRIGGER trg_check_exhibition_dates
BEFORE INSERT ON Exhibition
FOR EACH ROW
BEGIN
  IF NEW.endDate IS NOT NULL AND NEW.endDate <= NEW.startDate THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Erreur : endDate doit être postérieure à startDate.';
  END IF;
END$$
-- Même vérification à la mise à jour
CREATE TRIGGER trg_check_exhibition_dates_update
BEFORE UPDATE ON Exhibition
FOR EACH ROW
BEGIN
  IF NEW.endDate IS NOT NULL AND NEW.endDate <= NEW.startDate THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Erreur : endDate doit être postérieure à startDate.';
  END IF;
END$$
```

```sql
-- TRIGGER 2 : Bloquer une réservation si l'atelier est complet
CREATE TRIGGER trg_check_workshop_capacity
BEFORE INSERT ON Booking
FOR EACH ROW
BEGIN
  DECLARE current_count INT;
  SELECT COUNT(*) INTO current_count
  FROM Booking
  WHERE id_workshop = NEW.id_workshop
    AND paymentStatus != 'CANCELLED';

  IF current_count >= (
      SELECT maxParticipants FROM Workshop WHERE id_workshop = NEW.id_workshop
  ) THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Erreur : cet atelier est complet.';
  END IF;
END$$

-- TRIGGER 3 : Mettre à jour automatiquement le statut d'une œuvre à EXHIBITED
--             quand elle est associée à une exposition
CREATE TRIGGER trg_set_artwork_exhibited
AFTER INSERT ON Artwork_Exhibition
FOR EACH ROW
BEGIN
  UPDATE Artwork
  SET status = 'EXHIBITED'
  WHERE id_artwork = NEW.id_artwork
    AND status = 'FOR_SALE';
END$$

-- TRIGGER 4 : Table d'audit des modifications de prix des œuvres
CREATE TABLE IF NOT EXISTS Artwork_Price_Audit (
  id_audit    INT AUTO_INCREMENT PRIMARY KEY,
  id_artwork  INT,
  old_price   DECIMAL(15,2),
  new_price   DECIMAL(15,2),
  changed_at  DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TRIGGER trg_audit_artwork_price
BEFORE UPDATE ON Artwork
FOR EACH ROW
BEGIN
  IF NEW.price <> OLD.price THEN
    INSERT INTO Artwork_Price_Audit (id_artwork, old_price, new_price)
    VALUES (OLD.id_artwork, OLD.price, NEW.price);
  END IF;
END$$

DELIMITER ;
```

Procédures :

Procédure 1 :

```
-- PROCÉDURE 1 : Inscrire un membre à un atelier
--   Vérifie l'existence, les doublons, la capacité, puis crée la réservation.
CREATE PROCEDURE sp_inscrire_membre(
  IN p_id_member   INT,
  IN p_id_workshop INT,
  IN p_payment     VARCHAR(50)
)
BEGIN
  DECLARE v_exists_member   INT;
  DECLARE v_exists_workshop INT;
  DECLARE v_already         INT;
  DECLARE v_count           INT;
  DECLARE v_max             INT;

  SELECT COUNT(*) INTO v_exists_member   FROM CommunityMember WHERE id_member   = p_id_member;
  SELECT COUNT(*) INTO v_exists_workshop FROM Workshop         WHERE id_workshop = p_id_workshop;
  SELECT COUNT(*) INTO v_already         FROM Booking
    WHERE id_member = p_id_member AND id_workshop = p_id_workshop;

  IF v_exists_member = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Membre introuvable.';
```

```
ELSEIF v_exists_workshop = 0 THEN
  SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Atelier introuvable.';
ELSEIF v_already > 0 THEN
  SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Ce membre est déjà inscrit à cet atelier.';
END IF;

SELECT COUNT(*), maxParticipants
  INTO v_count, v_max
  FROM Booking b
  JOIN Workshop w ON b.id_workshop = w.id_workshop
  WHERE b.id_workshop = p_id_workshop AND b.paymentStatus != 'CANCELLED'
  GROUP BY w.maxParticipants;

IF v_count >= v_max THEN
  SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Atelier complet.';
END IF;

INSERT INTO Booking (bookingDate, paymentStatus, id_member, id_workshop)
VALUES (NOW(), p_payment, p_id_member, p_id_workshop);

SELECT 'Inscription réussie.' AS message;
```

Procédure 2 :

```sql
-- PROCÉDURE 2 : Créer une exposition et y associer automatiquement
--               les œuvres EXHIBITED de la galerie cible
CREATE PROCEDURE sp_creer_exposition(
  IN p_title       VARCHAR(200),
  IN p_startDate   DATE,
  IN p_endDate     DATE,
  IN p_theme       VARCHAR(100),
  IN p_curator     VARCHAR(100),
  IN p_id_gallery  INT
)
BEGIN
  DECLARE v_new_id INT;

  INSERT INTO Exhibition (title, startDate, endDate, theme, curatorName, id_gallery)
  VALUES (p_title, p_startDate, p_endDate, p_theme, p_curator, p_id_gallery);

  SET v_new_id = LAST_INSERT_ID();
  SELECT CONCAT('Exposition créée avec id = ', v_new_id) AS message;
END$$
```

Procédure 3 :

```sql
-- PROCÉDURE 3 : Annuler une réservation
CREATE PROCEDURE sp_annuler_reservation(
  IN p_id_booking INT
)
BEGIN
  IF NOT EXISTS (SELECT 1 FROM Booking WHERE id_booking = p_id_booking) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Réservation introuvable.';
  END IF;

  UPDATE Booking
  SET paymentStatus = 'CANCELLED'
  WHERE id_booking = p_id_booking;

  SELECT 'Réservation annulée.' AS message;
END$$
```

Fonctions stockées :

Fonction 1 :

```sql
-- FONCTION 1 : Retourner le nombre de participants confirmés à un atelier
CREATE FUNCTION fn_nb_participants(p_id_workshop INT)
RETURNS INT
DETERMINISTIC
READS SQL DATA
BEGIN
  DECLARE v_count INT;
  SELECT COUNT(*) INTO v_count
  FROM Booking
  WHERE id_workshop = p_id_workshop
    AND paymentStatus != 'CANCELLED';
  RETURN v_count;
END$$
```

Fonction 2 :

```sql
-- FONCTION 2 : Calculer la note moyenne d'un artiste (via ses œuvres)
CREATE FUNCTION fn_note_artiste(p_id_artist INT)
RETURNS DECIMAL(3,1)
DETERMINISTIC
READS SQL DATA
BEGIN
  DECLARE v_avg DECIMAL(3,1);
  SELECT ROUND(AVG(r.rating), 1) INTO v_avg
  FROM Review r
  JOIN Artwork a ON r.id_artwork = a.id_artwork
  WHERE a.id_artist = p_id_artist;
  RETURN COALESCE(v_avg, 0);
END$$
```

Fonction 3 :

```sql
-- FONCTION 3 : Vérifier si un membre peut encore s'inscrire à un atelier
CREATE FUNCTION fn_peut_sinscrire(p_id_member INT, p_id_workshop INT)
RETURNS BOOLEAN
DETERMINISTIC
READS SQL DATA
BEGIN
  DECLARE v_already  INT;
  DECLARE v_count    INT;
  DECLARE v_max      INT;

  SELECT COUNT(*) INTO v_already
  FROM Booking
  WHERE id_member = p_id_member AND id_workshop = p_id_workshop;

  SELECT COUNT(b.id_booking), w.maxParticipants
    INTO v_count, v_max
  FROM Booking b
  JOIN Workshop w ON b.id_workshop = w.id_workshop
  WHERE b.id_workshop = p_id_workshop
    AND b.paymentStatus != 'CANCELLED'
  GROUP BY w.maxParticipants;

  IF v_already > 0 OR COALESCE(v_count, 0) >= COALESCE(v_max, 0) THEN
    RETURN FALSE;
  END IF;
  RETURN TRUE;
END$$

DELIMITER ;
```

On inscrit un membre à plusieurs ateliers en une seule opération atomique, et on met à jour le type du membre s'il s'agit d'un membre premium. Si l'in des ateliers est complet ou indisponible, on procède à un rollback total :

```
CREATE PROCEDURE sp_inscription_multiple(
  IN p_id_member INT,
  IN p_workshop1 INT,
  IN p_workshop2 INT,
  IN p_workshop3 INT
)
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    SELECT 'Transaction annulée : une inscription a échoué. Aucun changement effectué.' AS erreur;
  END;

  START TRANSACTION;

  -- Inscription atelier 1
  IF fn_peut_sinscrire(p_id_member, p_workshop1) THEN
    INSERT INTO Booking (bookingDate, paymentStatus, id_member, id_workshop)
    VALUES (NOW(), 'CONFIRMED', p_id_member, p_workshop1);
  ELSE
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Impossible de s''inscrire à l''atelier 1.';
  END IF;

  -- Inscription atelier 2
  IF fn_peut_sinscrire(p_id_member, p_workshop2) THEN
    INSERT INTO Booking (bookingDate, paymentStatus, id_member, id_workshop)
    VALUES (NOW(), 'CONFIRMED', p_id_member, p_workshop2);
  ELSE
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Impossible de s''inscrire à l''atelier 2.';
  END IF;

  -- Inscription atelier 3
  IF fn_peut_sinscrire(p_id_member, p_workshop3) THEN
    INSERT INTO Booking (bookingDate, paymentStatus, id_member, id_workshop)
    VALUES (NOW(), 'CONFIRMED', p_id_member, p_workshop3);
  ELSE
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Impossible de s''inscrire à l''atelier 3.';
  END IF;
```

```sql
    -- Bonus : passer le membre en 'premium' s'il a 3 ateliers ou plus
    UPDATE CommunityMember
    SET membershipType = 'premium'
    WHERE id_member = p_id_member
      AND (SELECT COUNT(*) FROM Booking
           WHERE id_member = p_id_member
             AND paymentStatus = 'CONFIRMED') >= 3;

    COMMIT;
    SELECT 'Transaction réussie : 3 inscriptions enregistrées.' AS message;
  END$$

  DELIMITER ;
```

Scripts de test des transactions

```sql
-- Test des vues
SELECT * FROM v_catalogue_public;
SELECT * FROM v_programme_expositions;
SELECT * FROM v_ateliers_disponibilite;
SELECT * FROM v_profil_artiste;

-- Test des fonctions
SELECT fn_nb_participants(1)   AS inscrits_atelier_1;
SELECT fn_note_artiste(1)      AS note_da_vinci;
SELECT fn_peut_sinscrire(9, 1) AS membre9_peut_atelier1;

-- Test de la procédure d'inscription simple
CALL sp_inscrire_membre(9, 1, 'CONFIRMED');

-- Test de la transaction multiple (membres 9 et 10 sur 3 ateliers)
CALL sp_inscription_multiple(10, 1, 2, 3);

-- Test : inscription sur atelier complet (doit échouer si maxParticipants atteint)
-- CALL sp_inscrire_membre(10, 5, 'CONFIRMED');

-- Test trigger date invalide (doit échouer)
-- INSERT INTO Exhibition (title, startDate, endDate, theme, curatorName, id_gallery)
-- VALUES ('Test', '2026-06-01', '2026-01-01', 'Erreur', 'Test', 1);

-- Test annulation de réservation
CALL sp_annuler_reservation(1);

-- Vérification de l'audit de prix
UPDATE Artwork SET price = 999999999 WHERE id_artwork = 1;
SELECT * FROM Artwork_Price_Audit;
```